ANÁLISIS ARQUITECTÓNICO Y MODELO FORMAL DEL SISTEMA RESIPARK

Proyecto: Control Concurrente de Accesos **Autor:** Área de Ingeniería Informática **Evaluación:** IPC 1 - USAC (2º Semestre 2026)

1. RESUMEN EJECUTIVO Y ARQUITECTURA DE SOFTWARE

El presente informe documenta la especificación formal del software **ResiPark**, un sistema determinista orientado a objetos diseñado para la gestión de acceso vehicular, administración de espacios de parqueo y registro dinámico de residentes. El núcleo del sistema opera bajo la restricción estricta de abstención de colecciones genéricas provistas por el framework estándar, fundamentando la persistencia en memoria sobre algoritmos de asignación dinámicos mediante punteros expresos y estructuras lineales propias.

Modelo de Concurrencia y Consistencia Operacional:
La gestión simultánea de accesos se modela mediante sincronización en exclusión mutua. Sean G_1, G_2 los hilos receptores en la garita de entrada y Q la cola FIFO de contención, el estado global del espacio de parqueo S evoluciona mediante transiciones atómicas:

$$S_{t+1} = \delta(S_t, v_i), \quad \text{donde} \quad v_i = \text{desencolar}(Q) \quad \forall \quad Thread \in \{G_1, G_2\}$$

2. FORMALIZACIÓN DE ESTRUCTURAS DE DATOS HETEROGÉNEAS

Estructura	Topología de Enlace	Complejidad Temporal	Mecanismo de Control y Restricción
ListaDobleResidentes	Doble enlace (Cabecera / Cola)	Inserción: $O(1)$ Búsqueda: $O(n)$	Implementa iteración bidireccional estricta. Integra la función hash analítica `existePlacaGlobal()`.
ListaSimpleVehiculos	Enlace simple direccionado	Búsqueda: $O(k)$ $k \leq 3$	Acotada explícitamente a $k = 3$ unidades por nodo de residente. Eliminación sujeta a estado *FUERA*.
ListaCircularParqueo	Circular simétrica simple	Búsqueda: $O(N)$	Asignación rotativa determinista a partir del puntero P_{last}, garantizando equidad en la distribución de casillas.
ColaFIFO	Lineal mono-direccionada	Encolar/Desencolar: $O(1)$	Sincronización por monitores explícitos (`synchronized`) utilizando primitivas `wait()` y `notifyAll()`.
PilaEventos	LIFO dinámico	Apilar/Desapilar: $O(1)$	Mantiene la trazabilidad histórica de eventos bajo ordenamiento cronológico inverso en el tope.

3. SINCRONIZACIÓN Y SUBSUNCIÓN DE HILOS CONCURRENTES

El procesamiento en paralelo de las solicitudes de ingreso y egreso se ejecuta a través de tres subprocesos independientes derivados de `java.lang.Thread`. La preservación de la integridad del estado en el dominio de la interfaz gráfica Swing se delega de forma asíncrona al hilo de despacho de eventos (EDT) mediante invocaciones explícitas a `SwingUtilities.invokeLater()`, anulando condiciones de carrera (*race conditions*) y bloqueos mútuos (*deadlocks*).

4. DIAGRAMA ESTRUCTURAL DE CLASES (UML 2.5)

Modelo de dominio del sistema ResiPark. Muestra las clases de entidad, estructuras dinámicas personalizadas con abstracción de nodos, capas de concurrencia mediante herencia de subprocesos, persistencia en disco y la interfaz de usuario.

